Unsupervised learning and PCA

General idea

In **unsupervised learning** the **training examples** are **not associated with a label**

- We **observe data** from the **unknown distribution** $p_{\text{data}} \in \Delta(\mathcal{X})$
- Implicit supervision is given via the **design of the objective function**

We want to:

- **Simplify data** → **dimensionality reduction**, **clustering**, **density estimation**
- **Keep important information**

A great way to proceed is the **Principal Component Analysis (PCA)**

Principal Component Analysis

A technique that **allows to simplify data** while **keeping relevant information**
(can even be used as a preprocessing technique)

The idea behind is:

- **Find** the **direction of maximum variance**
- **Find** the **orthogonal directions**, capturing the **remaining variance**
- **Change** the **coordinate system** to **align** the **maximum variance** direction with the **main axe**
- **Apply dimensionality reduction** by **dropping** the **dimensions** of **least variance**

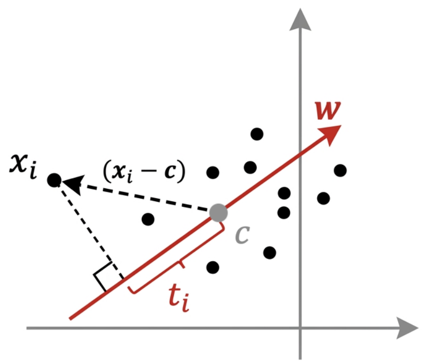
💡 "We **rotate the system** of coordinates to **view data from a better perspective**,
keeping only the directions that significantly matter"

The optimization problem

We want **to find the direction of max variance**

Geometric interpretation

Consider the **geometrical interpretation** of the problem:



Subject to $w^\top w = 1$ (which means $\|w\| = 1$) because:

- We want to **find the line that best fits** the **max variance direction**
- We should **ignore how long the vector is** and **focus** only **on** its **direction**
- → w must be a **unit vector**

Observe that:

$$t_i = (x_i - c)^\top w$$

Where:

- x_i is a **sample datapoint**
- c is the **mean of all** the **sample datapoint**
- w is the **unit direction**
- t_i is the **projection** of x_i **onto** the **line along which** w lies

It happens because:

- $(x_i - c)$ shifts the center to the mean of datapoints
- $(x_i - c)^\top w$ is the projection t_i of $(x_i - c)$ on w
  (as the matrix multiplication $\top$ indicates the dot product)

💡 We want to identify the direction for which the distance between the projections is maximal

MEAN VALUE OF PROJECTIONS

$$
\begin{aligned}
E[t] &= \frac{1}{n}\sum_{i=1}^{n} t_i = \frac{1}{n}\sum_{i=1}^{n} (x_i - c)^\top w \\
&= (\frac{1}{n}\sum_{i=1}^{n} x_i)^\top w - c^\top w \\
&= \bar{x}^\top w - c^\top w
\end{aligned}
$$

Where $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$

VARIANCE OF PROJECTIONS

$$
\begin{aligned}
Var[t] &= \frac{1}{n}\sum_{i=1}^{n} (t_i - E[t])^2 = \frac{1}{n}\sum_{i=1}^{n} [(x_i - \bar{x})^\top w]^2 \\
&= \frac{1}{n}\sum_{i=1}^{n} (\bar{x}_i{}^\top w)^2 = w^\top \underbrace{\left[\frac{1}{n}\sum_{i=1}^{n} \bar{x}_i \bar{x}_i^\top\right]}_{C} w
\end{aligned}
$$

We assume that $E[t] = 0$ (projections are already centered)

COVARIANCE MATRIX

We have observed that the variance can be written as:

$$Var[t] = w^\top C w$$

Where C is the covariance matrix

Recap: Variance and covariance

Variance and **covariance measure** the **spread of a set of point around** the **center of mass**

Variance

Measures the **deviation from** the **mean** for points in **one dimension**

Covariance

Measures how much each of the **dimensions vary from** the **mean**

- Respect to each other
- Is measured between two dimensions
- → **Allows** to **see** if there is a **correlation between two** dimensions

Recap: Eigenvalues and eigenvectors

Eigenvalues (= autovalori) and **eigenvectors** (= autovettori) **respect** the following rule:

$$\mathrm{A}x = \lambda x$$

Where:

- A is a **square matrix**
- x is an **eigenvector**
- λ is an **eigenvalue**

👀 We **must observe** that $\mathrm{A}x$ and λx are **two parallel vectors**

- **Same direction**
- **Different magnitude** (if $\mathrm{A} \neq \lambda$)

To find eigenvalues we have to **solve**:

$$\det(\mathrm{A} - \lambda I) = 0$$

Recap: Eigenvalue decomposition

We can **manipulate** the **variance matrix** C as:

$$C = U\Lambda U^\top = \sum_{j=1}^{m} \lambda_j \mathbf{u_j}\mathbf{u_j}^\top$$

Subject to $U^\top U = UU^\top = I$

Where:

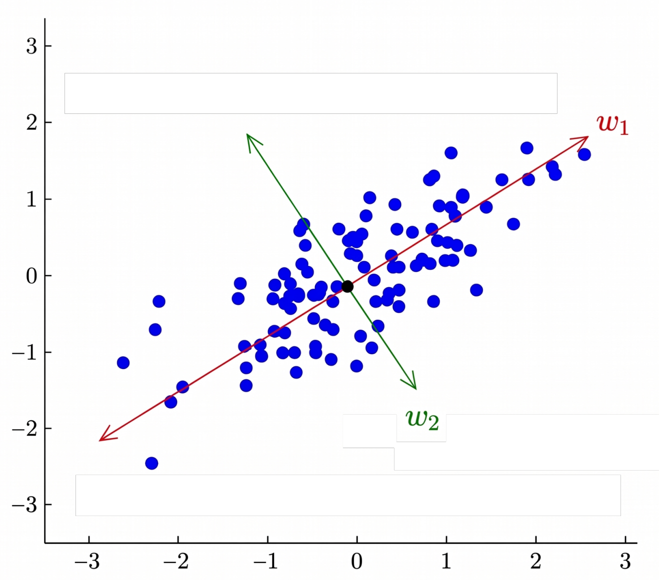
- $U = [u_1, \ldots, u_m] \in \mathbb{R}^{m \times m}$ where u_i is the eigenvector of the i-th component
- $\Lambda = [\lambda_1, \ldots, \lambda_m] \in \mathbb{R}^m$ where λ_i is the eigenvalue of the i-th component
  - $\lambda_1, \ldots, \lambda_m$ lie on the diagonal
  - We assume $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_m$ → eigenvalues are ordered from the highest to the lowest
- I is the identity matrix

💡 **Each component** represents **a specific feature of data**

- **As** well as **each eigenvector** → **together** they **represent** the sample datapoint

Optimization problem

We can proceed to **define the optimization problem**



We have observed that:

- We **want** to **maximize** the **variance** $Var[t]$
- We can **express** $Var[t]$ **as** $w^\top Cw$
- **Everything** is **subjected to** $w^\top w = 1$

We **define** the **function to maximize via Langrange multipliers**:

$$\mathscr{L}(w, \lambda) = w^\top Cw - \lambda(w^\top w - 1)$$

We **differentiate** $\mathscr{L}$ on w and set the result to 0 to **identify** the **max**:

$$\frac{d\mathscr{L}(w, \lambda)}{dw} = 2Cw - 2\lambda w = 0 \implies Cw = \lambda w$$

Moreover we observe that:

- C is a **square matrix**
- w is the **associated eigenvector**
- λ is the **associated eigenvalue**

As a consequence the **maximized variance can be expressed as** follows:

$$Var[t] = w^\top Cw = w^\top \lambda = \lambda \qquad \text{as } w^\top w = 1$$

and it is **expressed along** the **direction** of the **associated eigenvector** w

FIRST PRINCIPAL COMPONENT

We search for the unit vector w_1 that maximizes the variance

$$w_1 \in \arg\max\{w^\top Cw : w^\top w = 1\}$$

- The **largest eigenvalue** of C is the **variance along the first principal component**
- w_1 is the **corresponding eigenvector**, as a cons it is the **first principal component**

SECOND PRINCIPAL COMPONENT

$$w_2 \in \arg\max\{w^\top C w : w^\top w = 1, w \perp w_1\}$$

- The **second largest eigenvalue** of C is the **variance along the second principal component**
- w_2 is the **corresponding eigenvector**, as a cons it is the **first principal component**

i-TH PRINCIPAL COMPONENT

$$w_i \in \arg\max\{w^\top C w : w^\top w = 1, w \perp w_j \text{ for } 1 \leq j < i\}$$

- The **i-th largest eigenvalue** of C is the **variance along the i-th principal component**
- w_i is the **corresponding eigenvector**, as a cons it is the **i-th principal component**

Dimensionality reduction using PCA

Let $\hat{W} = [\mathbf{w_1}, \ldots, \mathbf{w_k}]$ hold the first k principal components
derived from data points $\bar{X} = [\bar{\mathbf{x}}_\mathbf{1}, \ldots, \bar{\mathbf{x}}_\mathbf{n}]$

Principal Component Scores

We change to a reduced coordinate system with the k principal components as axes:

$$T = \hat{W}^\top \bar{X} \in \mathbb{R}^{k \times n}$$

T is called principal component scores and computes
the new coordinates of samples in the new system

PCA using eigenvalue decomposition

Input: Data points $X = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]$

Centering: $\bar{X} = X - \boxed{\frac{1}{n}X1_n}1_n^\top$ → $\bar{x}$ mean datapoint; → Vector of ones

Compute covariance matrix: $C = \frac{1}{n}\bar{X}\bar{X}^\top$

Eigenvalue decomposition:

$$U, \lambda = \text{eig}(C)$$

Output: Principal components $W = U = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m]$ and variances $\lambda = (\lambda_1, \ldots, \lambda_m)$

PCA using singular value decomposition

Input: Data points $X = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]$

Centering: $\bar{X} = X - \boxed{\frac{1}{n}X1_n}1_n^\top$ → $\bar{x}$ mean datapoint; → Vector of ones

SVD decomposition:

$$U, \boldsymbol{s}, V = \text{SVD}(\bar{X})$$

Output: Principal components $U = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k]$ and variances

$$\left(\frac{s_1^2}{n}, \ldots, \frac{s_k^2}{n}\right)$$

Because $\bar{X} = USV^\top$ and $C = \frac{1}{n}\bar{X}\bar{X}^\top = \frac{1}{n}USV^\top VSU^\top = U\boxed{\frac{S^2}{n}}U^\top$

Scaling of variables

The **scale of features matters**

- It is recommended to scale them **to have unit standard deviation**

Number of components

The **number of principal components** depends on the **goal and application**

- **No mean to validate it** in unsupervised learning
- We can **compute** the **cumulative proportion of explained variance**
  - **Estimates** the amount of **information loss**
  - For the first principal component is:

Eigenvalue decomposition: $\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{m} C_{jj}}$ Singular value decomposition: $\frac{\sum_{j=1}^{k} s_j^2}{\sum_{ij} \bar{X}_{ji}^2}$

Kernel PCA

PCA reduces the **dimensionality via** a **linear transformation**

- What to do **if data doesn't follow a linear distribution**?
- → **We can use the kernel trick**
  - To **cope** with a **non-linear transformation** in the original space